

Working with Git for Windows

Version control systems play an important role in tracking changes in software and enable users to fall back to a desired previous version. Git is a distributed version control system that can be run on Windows, the mechanics of which are demonstrated in this article.

Version control software is that which helps software teams to manage changes in source code over time. It stores these changes in a special database. A group of changes is stored in a special type of database called a commit. If new changes made by developers are not acceptable for whatever reason, they can switch back to the original version, using version control methods.

Currently, the most widely used version control system is Git, which is an actively maintained open source project by Linus Torvalds, the creator of the Linux OS. Git is an example of a distributed version control system. Rather than have a single location to maintain the history of development, each developer has a code base which is her own repository, from which she can choose what to send to the server and what remains with her. Git provides a great amount of performance, security and flexibility in version maintenance.

Installing Git on Windows

To install Git on Windows, download the Git installer for Windows from <https://git-for-windows.github.io>. Run the executable to complete the installation. The default options are pretty good for many users. Open *Git Bash* from the *Start* menu. To work with Git, you need to set up your local Git username and email ID. To set the username, use the following command:

```
git config --global user.name "my_username"
```

To set the email ID, use the following command:

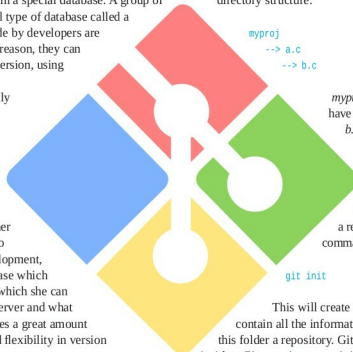
```
git config --global user.email "my_email"
```

After this step, Git is completely ready to be used and you can now locally start managing your source code.

The Git repository

Git repositories are folders of your source code, in which

the history of the changes made in the code is stored. For example, let's look at a small project that has the following directory structure:



myproj is the directory, where I have my source code and *a.c* and *b.c* are my source files, which need to be tracked. First, through *Git bash*, go to the folder. We need to make Git aware of this directory—that is, we need to make this folder a repository; so type the following command to make this happen:

```
git init
```

This will create a *.git* folder, which will contain all the information that Git needs to make this folder a repository. Git commands can only be run inside a Git repository, so *git init* is the first step.

Note: You can also run *git init* on an empty folder to make it a Git repository, and can start adding files to it.

Running *git init* will not modify any files in the source code directory. *Git init* is run only when the project is starting; once initialisation of the Git repository is complete, this project can be hosted on the Git server. Other developers can clone this repository.

Cloning in Git means copying an already existing Git repository from some server. The repository that gets copied is a fully functional Git repository—it has its own history, its own files and is totally isolated from the original repository from which it was cloned. The following is the command to clone a Git repository:

```
git clone <repo_path>
```

Here, *repo_path* is the URL of the Git repository on the server.